Software Engineering Project May - 2026

Milestone 4 Deliverables for Neighbourly

Submitted By

Team Name: BugsBunny

Team Code: Team-015



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

Apartment Association Management System

Project Details

Software Name: Neighbourly - Apartment Association Management System

Date of Submission: 12-08-2026

Team Details

Team Name: BugsBunny
Team Code: Team-015

Name	Roll Number	Role *
YASH KARSH	22f3002322	Project Architect
Juhi Parekh	23f2005473	Scrum Master
SAFWAN HUMAYUN	24f1001022	Frontend Developer
SANIYA NAAZ	23f2002592	Backend Developer
Saumya Sarkar	21f3001551	Backend Developer/Software Tester

*These are the team's primary roles, and members will collaborate across tasks as the project continues.

Executive Summary

This Milestone 4 report presents the API implementation, validation, user feedback, and implemented improvements for Neighbourly. The system was evaluated using a hybrid strategy combining direct pytest-backed validation and Swagger-driven contract validation.

At the latest stage of validation, the system covered 97 OpenAPI operations across 10 backend modules, with 103 passing test cases, 0 failing test cases, and 0 unverified operations. All 97 endpoints were validated through direct pytest evidence. The direct evidence set included 103 test cases covering happy-path flows, role-based access control (RBAC), input validation, business rule enforcement, and error handling.

Earlier execution reporting covered the transition from Milestone 3, where the system had 30 endpoints across 7 modules. Milestone 4 added 5 new modules (Billing & Payment, Communication & Notification, Community Engagement, Maintenance Staff Management, and AI Community Intelligence), expanding the API surface by 160%.

In addition to technical validation, user feedback was collected from three key roles: Resident, Admin/Committee, and Maintenance Staff. Based on this feedback, the system was updated with billing automation, unified notifications, community polling, AI-assisted complaint classification, and a complete maintenance task lifecycle. A real payment gateway integration and SMS/email notifications remain pending for future sprints.

API Validation Metrics

API Validation Metrics	
Metric	**Value**
Total API Endpoints	97
Modules Implemented	10
Total Test Cases	103
Passing Test Cases	103
Failing Test Cases	0
Unverified Operations	0
Direct pytest Coverage	103 test cases across 12 test files
Swagger/OpenAPI Paths	78 paths (OpenAPI 3.1.0)
Codebase Source Files	65 Python modules
Bug Fixes Applied	11 (documented in code comments)
User Stories Covered	US-01 through US-15 + AI features

Validation Approach

1. **Direct pytest Validation:** 103 async test cases using httpx.AsyncClient + SQLite in-memory database. Each test authenticates, exercises an endpoint, and asserts status code, response shape, and database state.

2. **Swagger-Driven Probe Validation:** OpenAPI 3.1.0 spec (api-documentation/swagger.yaml) defines all 97 endpoints with schemas, security requirements, and error responses.

3. **Contract Checks:** Request/response schemas validated against Pydantic models; RBAC enforced via FastAPI dependencies.

User Story to API Mapping

User Story to API Mapping		
User Story	**Description**	**API Endpoints**
US-01	Resident views maintenance request status	GET /api/v1/complaints, GET /api/v1/complaints/{id}
US-02	Resident reserves shared facilities	POST /api/v1/admin/facilities, POST /api/v1/facility-bookings, GET /api/v1/facility-bookings
US-03	Resident pre-registers visitors	POST /api/v1/visitors/pre-register, GET /api/v1/visitors/requests
US-04	Resident receives notifications	GET /api/v1/notifications, POST /api/v1/notifications/{id}/read, POST /api/v1/notifications/read-all
US-05	Resident views announcements	GET /api/v1/announcements, GET /api/v1/announcements/{id}
US-06	Admin assigns maintenance requests	POST /api/v1/complaints/{id}/assign, GET /api/v1/admin/maintenance-staff
US-07	Admin publishes announcements	POST /api/v1/announcements, PATCH /api/v1/announcements/{id}, DELETE /api/v1/announcements/{id}
US-08	Maintenance staff views assigned tasks	GET /api/v1/maintenance/tasks, POST /api/v1/maintenance/tasks/{id}/accept, POST /api/v1/maintenance/tasks/{id}/start, POST /api/v1/maintenance/tasks/{id}/complete
US-09	Security guard verifies visitor details	POST /api/v1/visitors/{request_id}/verify-otp
US-10	Security guard records entry/exit	POST /api/v1/visitors/{request_id}/verify-otp (entry), POST /api/v1/visitors/logs/{log_id}/exit (exit)
US-11	Resident manages visitor pre-registration	POST /api/v1/visitors/pre-register, GET /api/v1/visitors/requests
US-12	Admin manages complaint categories	POST /api/v1/admin/complaint-categories, GET /api/v1/admin/complaint-categories
US-13	Resident files complaints with images	POST /api/v1/complaints, POST /api/v1/complaints/{id}/images
US-14	Vendor service visit logged	POST /api/v1/visitors/pre-register (vendor type), POST /api/v1/visitors/{request_id}/verify-otp, POST /api/v1/visitors/logs/{log_id}/exit
US-15	Security guard verifies visitor OTP	POST /api/v1/visitors/{request_id}/verify-otp
US-16	Admin views billing summary	GET /api/v1/admin/bills/summary, GET /api/v1/admin/bills
US-17	Resident views and pays bills	GET /api/v1/billing/me/bills, POST /api/v1/payments, GET /api/v1/payments/{id}/receipt
US-18	Resident participates in polls	GET /api/v1/polls, POST /api/v1/polls/{id}/vote, GET /api/v1/polls/{id}/results
US-19	AI-assisted complaint classification	POST /api/v1/ai/complaints/classify
US-20	AI staff recommendations	GET /api/v1/ai/complaints/{id}/staff-recommendations
US-21	AI smart search	GET /api/v1/ai/search
US-22	AI complaint summary report	POST /api/v1/ai/reports/complaints/summary
US-23	AI community chat	POST /api/v1/ai/chat

API Description by Module

Module 1 - Authentication & Authorization				
#	Method	Path	Auth	Description
1	POST	/api/v1/auth/register-committee	Admin	Create Committee account
2	POST	/api/v1/auth/signup	Public	Self-registration (Resident/Maintenance/Security)
3	POST	/api/v1/auth/login	Public	Login with access + refresh tokens
4	POST	/api/v1/auth/refresh	Public (token)	Refresh access token
5	POST	/api/v1/auth/logout	Authenticated	Revoke refresh token
6	POST	/api/v1/auth/forgot-password	Public	Initiate password reset
7	POST	/api/v1/auth/reset-password	Public (token)	Complete password reset
8	GET	/api/v1/auth/me	Authenticated	Get current user profile
9	GET	/api/v1/auth/pending-verifications	Committee	List unverified accounts
10	POST	/api/v1/auth/verify/{user_id}	Committee	Verify user (triggers move-in if resident)
11	GET	/api/v1/users	Admin/Committee	List all users
12	POST	/api/v1/users/{user_id}/deactivate	Admin/Committee	Deactivate user
13	POST	/api/v1/users/{user_id}/reactivate	Admin/Committee	Reactivate user
14	POST	/api/v1/auth/users/{user_id}/deactivate	Admin	Admin-only deactivate (stronger action)
15	POST	/api/v1/auth/users/{user_id}/reactivate	Admin	Admin-only reactivate
16	POST	/api/v1/residents/me/profile-picture	Resident	Upload profile picture
17	DELETE	/api/v1/residents/me/profile-picture	Resident	Delete profile picture
New in M4: Profile picture management, user deactivation/reactivation, pending verifications.				

Module 2 - User & Apartment Management				
#	Method	Path	Auth	Description
18	POST	/api/v1/blocks	Admin/Committee	Create block
19	GET	/api/v1/blocks	Public	List blocks
20	GET	/api/v1/blocks/{block_id}	Public	Get block details
21	PATCH	/api/v1/blocks/{block_id}	Admin/Committee	Update block
22	DELETE	/api/v1/blocks/{block_id}	Admin/Committee	Delete block
23	POST	/api/v1/flats	Admin/Committee	Create flat
24	GET	/api/v1/flats	Public	List flats (filters: block_id, flat_number, occupancy_status)
25	GET	/api/v1/flats/{flat_id}	Authenticated	Get flat with current residents
26	PATCH	/api/v1/flats/{flat_id}	Admin/Committee	Update flat
27	GET	/api/v1/residents	Admin/Committee	List residents (filters: block_id, flat_id)
28	GET	/api/v1/residents/{resident_id}	Admin/Committee	Get resident profile + flat/block
29	PATCH	/api/v1/residents/{resident_id}	Admin/Committee	Update resident profile
30	POST	/api/v1/residents/{resident_id}/move-in	Admin/Committee	Move resident into flat
31	POST	/api/v1/residents/{resident_id}/move-out	Admin/Committee	Move resident out of flat
32	GET	/api/v1/residents/{resident_id}/occupancy-history	Admin/Committee	Get occupancy history
Evolution from M3: Added flat detail endpoint, resident directory CRUD, move-in/move-out workflow, occupancy history tracking.				

Module 3 - Complaint Management				
#	Method	Path	Auth	Description
33	POST	/api/v1/admin/complaint-categories	Admin/Committee	Create complaint category
34	GET	/api/v1/admin/complaint-categories	Admin/Committee	List all categories
35	GET	/api/v1/complaint-categories	Authenticated	Public category list
36	POST	/api/v1/complaints	Verified Resident	File complaint
37	GET	/api/v1/complaints	Authenticated	List complaints (role-scoped)
38	GET	/api/v1/complaints/{complaint_id}	Authenticated	Get complaint details
39	PATCH	/api/v1/complaints/{complaint_id}	Authenticated	Update complaint (status transition enforced)
40	POST	/api/v1/complaints/{complaint_id}/assign	Admin/Committee	Assign to staff
41	POST	/api/v1/complaints/{complaint_id}/comments	Authenticated	Add feedback/rating
42	POST	/api/v1/complaints/{complaint_id}/images	Authenticated	Upload complaint image
Evolution from M3: Added admin complaint category CRUD, complaint images, status state machine (open → assigned → in_progress → resolved → closed), role-scoped complaint listing.				

Module 4 - Maintenance Staff Management				
#	Method	Path	Auth	Description
43	GET	/api/v1/admin/maintenance-staff	Admin/Committee	List staff + open_task_count
44	GET	/api/v1/maintenance/tasks	Verified Maintenance	List my assigned tasks
45	GET	/api/v1/maintenance/tasks/{task_id}	Verified Maintenance	Get task details (403 if not assigned)
46	POST	/api/v1/maintenance/tasks/{task_id}/accept	Verified Maintenance	Accept task
47	POST	/api/v1/maintenance/tasks/{task_id}/decline	Verified Maintenance	Decline task
48	POST	/api/v1/maintenance/tasks/{task_id}/schedule	Verified Maintenance	Schedule task
49	POST	/api/v1/maintenance/tasks/{task_id}/start	Verified Maintenance	Start task (in_progress)
50	POST	/api/v1/maintenance/tasks/{task_id}/complete	Verified Maintenance	Complete task
New in M4: Full maintenance task lifecycle (accept/decline/schedule/start/complete), ownership enforcement, automatic complaint status transitions.				

Module 5 - Visitor Management				
#	Method	Path	Auth	Description
51	POST	/api/v1/visitors/pre-register	Verified Resident	Pre-register visitor (returns OTP)
52	GET	/api/v1/visitors/requests	Verified Resident	List my requests
53	POST	/api/v1/visitors/{request_id}/verify-otp	Verified Security	Verify OTP at gate
54	POST	/api/v1/visitors/logs/{log_id}/exit	Verified Security	Log exit time
55	GET	/api/v1/visitors/logs	Admin	List all visitor logs
56	POST	/api/v1/visitors/{request_id}/deny	Verified Resident	Deny a visitor request before arrival
Evolution from M3: Added deny endpoint, admin-only visitor logs, timing-safe OTP hashing (HMAC-SHA256).				

Module 6 - Facility Booking				
#	Method	Path	Auth	Description
57	POST	/api/v1/admin/facilities	Admin/Committee	Create facility
58	GET	/api/v1/admin/facilities	Authenticated	List facilities
59	GET	/api/v1/admin/facilities/{facility_id}	Admin/Committee	Get facility details
60	PATCH	/api/v1/admin/facilities/{facility_id}	Admin/Committee	Update facility
61	POST	/api/v1/facility-bookings	Verified Resident	Book facility
62	GET	/api/v1/facility-bookings	Authenticated	List bookings (resident: own only)
63	GET	/api/v1/facility-bookings/{booking_id}	Authenticated	Get booking details
64	PATCH	/api/v1/facility-bookings/{booking_id}	Admin/Committee	Update/cancel booking
Evolution from M3: Added admin facility CRUD, overlapping slot detection (409), end-before-start validation (400), maximum duration enforcement, resident-scoped booking lists.				

Module 7 - Communication & Notification				
#	Method	Path	Auth	Description
65	POST	/api/v1/announcements	Admin/Committee	Create announcement (notifies all residents)
66	GET	/api/v1/announcements	Authenticated	List announcements (live only for non-admin)
67	GET	/api/v1/announcements/{announcement_id}	Authenticated	Get announcement details
68	PATCH	/api/v1/announcements/{announcement_id}	Admin/Committee	Update announcement
69	DELETE	/api/v1/announcements/{announcement_id}	Admin/Committee	Delete announcement
70	GET	/api/v1/notifications	Authenticated	List my notifications
71	POST	/api/v1/notifications/{notification_id}/read	Authenticated	Mark as read
72	POST	/api/v1/notifications/read-all	Authenticated	Mark all as read
New in M4: Full communication module. Announcements auto-generate notifications for all residents. Notification types: announcement, bill_generated, payment_success, complaint_update, booking_update, visitor_arrival, poll.				

Module 8 - Billing & Payment				
#	Method	Path	Auth	Description
73	POST	/api/v1/admin/bills	Admin/Committee	Generate bill
74	GET	/api/v1/admin/bills/summary	Admin/Committee	Billing summary (total_billed, total_collected, total_outstanding)
75	GET	/api/v1/admin/bills	Admin/Committee	List bills
76	GET	/api/v1/admin/bills/{bill_id}	Admin/Committee	Get bill details
77	PATCH	/api/v1/admin/bills/{bill_id}	Admin/Committee	Update bill
78	GET	/api/v1/billing/me/bills	Resident	Get my bills
79	GET	/api/v1/billing/me/payments	Resident	Get my payments
80	POST	/api/v1/payments	Verified Resident/ Admin/Committee	Record payment
81	GET	/api/v1/payments	Authenticated	List payments (role-scoped)
82	GET	/api/v1/payments/{payment_id}	Authenticated	Get payment details
83	PATCH	/api/v1/payments/{payment_id}	Verified Resident/ Admin/Committee	Update payment status
84	GET	/api/v1/payments/{payment_id}/receipt	Authenticated	Get payment receipt
New in M4: Complete billing lifecycle. Money handled as Decimal (Numeric(10,2)). Auto-generated receipt numbers (RCPT-{year}-{payment_id:06d}). Duplicate transaction reference protection (409).				

Module 9 - Community Engagement				
#	Method	Path	Auth	Description
85	POST	/api/v1/polls	Admin/Committee	Create poll (min 2 options)
86	GET	/api/v1/polls	Authenticated	List polls
87	GET	/api/v1/polls/{poll_id}	Authenticated	Get poll details
88	PATCH	/api/v1/polls/{poll_id}	Admin/Committee	Update poll
89	POST	/api/v1/polls/{poll_id}/vote	Verified Resident	Cast vote
90	GET	/api/v1/polls/{poll_id}/results	Authenticated	Get live results
New in M4: Poll creation, voting, results. Enforces minimum 2 options, active poll check, date range validation, single-vote enforcement.				

Module 10 - AI Community Intelligence				
#	Method	Path	Auth	Description
91	POST	/api/v1/ai/complaints/classify	Verified Resident/ Admin/Committee	AI classifies complaint (category + priority + confidence)
92	GET	/api/v1/admin/maintenance-staff	Admin/Committee	List staff with open_task_count (AI-enriched)
93	GET	/api/v1/ai/complaints/{complaint_id}/staff-recommend	Admin/Committee	AI-ranked staff recommendations
94	GET	/api/v1/ai/search	Authenticated	AI-enhanced search (complaints + announcements)
95	POST	/api/v1/ai/reports/complaints/summary	Admin/Committee	AI-generated complaint summary
96	POST	/api/v1/ai/chat	Authenticated	AI chatbot with role-scoped context
New in M4: AI module powered by Groq. Toggle via AI_ENABLED / GROQ_API_KEY. Returns 503 when disabled. Rate-limited chat (Redis). Cached summaries.				

Health & Utility				
#	Method	Path	Auth	Description
97	GET	/health	Public	Health check

Testing Strategy

The testing strategy combines direct pytest validation with contract-based verification to ensure API correctness, security, and alignment with user stories.

Layers:

1. **Unit/Integration Tests (pytest + httpx)**
   - Async test client hitting the actual FastAPI app
   - SQLite in-memory database for isolation
   - seeded_db fixture pre-loads roles and test data
   - 103 test cases across 12 files

2. **Contract Tests**
   - OpenAPI 3.1.0 spec defines expected request/response shapes
   - Pydantic schemas enforce validation at runtime
   - RBAC enforced via FastAPI dependency injection

3. **Error Path Tests**
   - 401 Unauthorized (missing/invalid token)
   - 403 Forbidden (insufficient role)
   - 404 Not Found (non-existent resource)
   - 409 Conflict (duplicate/overlap)
   - 422 Validation Error (Pydantic)

- 503 Service Unavailable (AI disabled)

Test Organisation

Test Organisation			
Test File	**Module**	**Test Cases**	**Focus**
test_auth.py	Module 1	17	Login, signup, RBAC, profile pictures, verification
test_apartments.py	Module 2	15	Blocks, flats, residents, move-in/move-out
test_complaints.py	Module 3	10	File, list, assign, comment, categories
test_maintenance.py	Module 4	8	Task accept/decline/schedule/start/complete, ownership
test_visitors.py	Module 5	7	Pre-register, OTP verify, exit logging, logs
test_facilities.py	Module 6	12	Booking creation, overlap detection, cancellation
test_billing.py	Module 8	12	Bills, payments, receipts, duplicate protection
test_communication.py	Module 7	8	Announcements, notifications, read-all
test_community.py	Module 9	7	Polls, voting, results, inactive poll rejection
test_ai.py	Module 10	6	Classify (disabled/enabled), staff recs, search, chat
test_full_coverage.py	All	29	Missing endpoint coverage, edge cases

Execution

```bash
cd milestone-4/test-cases
python -m pytest -v
# Result: 103 passed, 0 failed
```

API Test Cases

Test Case 1 - Resident Files Complaint

User Story: US-01 (View maintenance status) + US-13 (File complaint)	
Field	**Value**
API	POST /api/v1/complaints
Test ID / Evidence	test_complaints.py::TestComplaints::test_resident_can_file_complaint
Input Payload	{"category_id": 1, "description": "Water leaking from ceiling", "priority": "high"}
Expected Result	200 OK, {"success": true, "data": {"status": "open", ...}}
Actual Result	200 OK, complaint created with status="open", resident_id set to caller
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_resident_can_file_complaint(self, client: AsyncClient, seeded_db: Session):
    _, profile, token = create_test_resident(seeded_db, flat_number="Comp-101", email="comp@test.com")
    category = ComplaintCategory(category_name="Plumbing")
    seeded_db.add(category)
    seeded_db.commit()
    seeded_db.refresh(category)
    response = await client.post(
        "/api/v1/complaints",
        headers={"Authorization": f"Bearer {token}"},
        json={"category_id": category.category_id, "description": "Water leaking from ceiling", "priority": "high"},
    )
    assert response.status_code == status.HTTP_200_OK
    data = response.json()
    assert data["data"]["description"] == "Water leaking from ceiling"
    assert data["data"]["status"] == "open"
    assert data["data"]["resident_id"] == profile.resident_id
```

Test Case 2 - Admin Assigns Complaint

User Story: US-06 (Admin assigns maintenance)	
Field	**Value**
API	POST /api/v1/complaints/{complaint_id}/assign
Test ID / Evidence	test_complaints.py::TestComplaints::test_admin_can_assign_complaint
Input Payload	{"staff_id": 1}
Expected Result	200 OK, {"success": true, "data": {"assignment_id": 1, "status": "assigned"}}
Actual Result	200 OK, ComplaintAssignment created, parent complaint status flipped to assigned
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_admin_can_assign_complaint(self, client: AsyncClient, seeded_db: Session):
    _, profile, resident_token = create_test_resident(seeded_db, flat_number="AssignComp-101", email="assigncomp@test.com")
    _, admin_token = create_test_user(seeded_db, RoleEnum.ADMIN.name, email="admin_assign@test.com")
    category = ComplaintCategory(category_name="General")
    seeded_db.add(category)
    seeded_db.commit()
    seeded_db.refresh(category)
    complaint = Complaint(
        resident_id=profile.resident_id,
        category_id=category.category_id,
        description="Broken door",
        priority="medium",
        status="open",
    )
```

```
    seeded_db.add(complaint)
    seeded_db.commit()
    seeded_db.refresh(complaint)
    maint_user, _ = create_test_user(seeded_db, RoleEnum.MAINTENANCE.name, email="maint_assign@test.com")
    staff = MaintenanceStaff(user_id=maint_user.user_id, specialization="General", experience_years=2, is_available=True)
    seeded_db.add(staff)
    seeded_db.commit()
    seeded_db.refresh(staff)
    response = await client.post(
        f"/api/v1/complaints/{complaint.complaint_id}/assign",
        headers={"Authorization": f"Bearer {admin_token}"},
        json={"staff_id": staff.staff_id},
    )
    assert response.status_code == status.HTTP_200_OK
    assert response.json()["data"]["assignment_id"] is not None
```

Test Case 3 - Overlapping Booking Rejected

User Story: US-02 (Reserve facilities)	
Field	**Value**
API	POST /api/v1/facility-bookings
Test ID / Evidence	test_facilities.py::TestFacilityBookings::test_booking_rejects_overlapping_slots
Input Payload	{"facility_id": 1, "booking_date": "2026-12-01", "start_time": "09:30:00", "end_time": "10:30:00"} (Previous booking exists for same facility on same date from 09:00–10:00)
Expected Result	409 Conflict
Actual Result	409 Conflict, {"success": false, "message": "This time slot overlaps with an existing booking."}
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_booking_rejects_overlapping_slots(self, client: AsyncClient, seeded_db: Session):
    block = Block(block_name="OLAP Block", total_flats=100)
    seeded_db.add(block)
    seeded_db.commit()
    seeded_db.refresh(block)
    flat = Flat(flat_number="OLAP-101", floor_number=1, occupancy_status="occupied", block_id=block.block_id)
    seeded_db.add(flat)
    seeded_db.commit()
    seeded_db.refresh(flat)
    facility = Facility(
        facility_name="Gym",
        description="",
        capacity=5,
        opening_time=time(6, 0),
        closing_time=time(22, 0),
        slot_duration=30,
        maximum_duration=60,
        is_active=True,
    )
```

```
    seeded_db.add(facility)
    seeded_db.commit()
    seeded_db.refresh(facility)
    _, profile, token = create_test_resident(seeded_db, flat_number="OLAP-101")
    await client.post(
        "/api/v1/facility-bookings",
        headers={"Authorization": f"Bearer {token}"},
        json={"facility_id": facility.facility_id, "booking_date": "2026-12-01", "start_time": "09:00:00", "end_time": "10:00:00"},
    )
    response = await client.post(
        "/api/v1/facility-bookings",
        headers={"Authorization": f"Bearer {token}"},
        json={"facility_id": facility.facility_id, "booking_date": "2026-12-01", "start_time": "09:30:00", "end_time": "10:30:00"},
    )
    assert response.status_code == status.HTTP_409_CONFLICT
```

Test Case 4 - Wrong OTP Rejected

User Story: US-09 (Verify visitor details)	
Field	**Value**
API	POST /api/v1/visitors/{request_id}/verify-otp
Test ID / Evidence	test_visitors.py::TestVisitors::test_wrong_otp_is_rejected
Input Payload	{"otp": "999999"} (Correct OTP is different; stored as HMAC-SHA256 hash)
Expected Result	400 Bad Request
Actual Result	400 Bad Request, {"success": false, "message": "Invalid or expired OTP."}
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_wrong_otp_is_rejected(self, client: AsyncClient, seeded_db: Session):
    _, profile, _ = create_test_resident(seeded_db, flat_number="WrongOTP-101", email="wrongotp@test.com")
    guard_user, guard_token = create_test_user(seeded_db, RoleEnum.SECURITY.name, email="guard_wrong@test.com")
    guard = SecurityGuard(user_id=guard_user.user_id, gate_assigned="Gate A", shift="Day")
    seeded_db.add(guard)
    seeded_db.commit()
    visitor = Visitor(visitor_name="Wrong Guest", phone="6666666666", visitor_type="guest")
    seeded_db.add(visitor)
    seeded_db.commit()
    seeded_db.refresh(visitor)
    request = VisitorRequest(
        resident_id=profile.resident_id,
        visitor_id=visitor.visitor_id,
        expected_date=date(2025, 8, 1),
        expected_time=time(14, 0),
        status="expected",
        otp="hashed_wrong_otp",
    )
    seeded_db.add(request)
    seeded_db.commit()
    seeded_db.refresh(request)
    response = await client.post(
        f"/api/v1/visitors/{request.request_id}/verify-otp",
        headers={"Authorization": f"Bearer {guard_token}"},
        json={"otp": "999999"},
    )
    assert response.status_code == status.HTTP_400_BAD_REQUEST
```

Test Case 5 - AI Classification Disabled

User Story: US-19 (AI-assisted classification)	
Field	**Value**
API	POST /api/v1/ai/complaints/classify
Test ID / Evidence	test_ai.py::TestAI::test_classify_disabled_returns_503
Input Payload	{"description": "Water leakage in bathroom ceiling"} (AI_ENABLED=false in test settings)
Expected Result	503 Service Unavailable
Actual Result	503 Service Unavailable, {"success": false, "message": "AI features are currently disabled."}
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_classify_disabled_returns_503(self, client: AsyncClient, seeded_db, monkeypatch):
    monkeypatch.setattr(settings, "AI_ENABLED", False)
    _user, _profile, token = create_test_resident(seeded_db)
    resp = await client.post(
        "/api/v1/ai/complaints/classify",
        headers={"Authorization": f"Bearer {token}"},
        json={"description": "Water leakage in bathroom ceiling"},
    )
    assert resp.status_code == status.HTTP_503_SERVICE_UNAVAILABLE
    body = resp.json()
    assert body["success"] is False
```

Test Case 6 - Admin Creates Bill

User Story: US-16 (Admin views billing summary)	
Field	**Value**
API	POST /api/v1/admin/bills
Test ID / Evidence	test_billing.py::TestBilling::test_admin_can_create_bill
Input Payload	{"flat_id": 1, "due_date": "2025-08-15", "amount": "2500.00", "billing_month": "2025-07"}
Expected Result	200 OK, {"success": true, "data": {"status": "unpaid", ...}}
Actual Result	200 OK, bill created with status="unpaid", flat_id set correctly
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_admin_can_create_bill(self, client: AsyncClient, seeded_db: Session):
    _, admin_token = create_test_user(seeded_db, RoleEnum.ADMIN.name, email="admin_bill@test.com")
    block = Block(block_name="Bill Block", total_flats=100)
    seeded_db.add(block)
    seeded_db.commit()
    seeded_db.refresh(block)
    flat = Flat(flat_number="Bill-101", floor_number=1, occupancy_status="occupied", block_id=block.block_id)
    seeded_db.add(flat)
    seeded_db.commit()
    seeded_db.refresh(flat)
    response = await client.post(
        "/api/v1/admin/bills",
        headers={"Authorization": f"Bearer {admin_token}"},
        json={
            "flat_id": flat.flat_id,
            "due_date": "2025-08-15",
            "amount": "2500.00",
            "billing_month": "2025-07",
        },
    )
    assert response.status_code == status.HTTP_200_OK
    data = response.json()
    assert data["data"]["flat_id"] == flat.flat_id
    assert data["data"]["status"] == "unpaid"
```

Test Case 7 - Resident Records Payment

User Story: US-17 (Resident views and pays bills)	
Field	**Value**
API	POST /api/v1/payments
Test ID / Evidence	test_billing.py::TestBilling::test_resident_can_record_payment
Input Payload	{"bill_id": 1, "amount": "1500.00", "payment_method": "upi", "transaction_reference": "txn_001", "status": "success"}
Expected Result	200 OK, {"success": true, "data": {"status": "success", ...}}
Actual Result	200 OK, payment recorded with status="success", auto-generated receipt
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_resident_can_record_payment(self, client: AsyncClient, seeded_db: Session):
    _, admin_token = create_test_user(seeded_db, RoleEnum.ADMIN.name, email="admin_payment@test.com")
    block = Block(block_name="Pay Block", total_flats=100)
    seeded_db.add(block)
    seeded_db.commit()
    seeded_db.refresh(block)
    flat = Flat(flat_number="Pay-101", floor_number=1, occupancy_status="occupied", block_id=block.block_id)
    seeded_db.add(flat)
    seeded_db.commit()
    seeded_db.refresh(flat)
    bill_resp = await client.post(
        "/api/v1/admin/bills",
        headers={"Authorization": f"Bearer {admin_token}"},
        json={"flat_id": flat.flat_id, "due_date": "2025-08-15", "amount": "1500.00", "billing_month": "2025-07"},
    )
    assert bill_resp.status_code == status.HTTP_200_OK
    bill_id = bill_resp.json()["data"]["bill_id"]
    _, profile, resident_token = create_test_resident(seeded_db, flat_number="Pay-101", email="payer@test.com")
    response = await client.post(
        "/api/v1/payments",
        headers={"Authorization": f"Bearer {resident_token}"},
        json={"bill_id": bill_id, "amount": "1500.00", "payment_method": "upi", "transaction_reference": "txn_001", "status": "success"},
    )
    assert response.status_code == status.HTTP_200_OK
    data = response.json()
    assert data["data"]["status"] == "success"
```

Test Case 8 - Committee Creates Announcement

User Story: US-07 (Admin publishes announcements)	
Field	**Value**
API	POST /api/v1/announcements
Test ID / Evidence	test_communication.py::TestCommunication::test_committee_can_create_announcement
Input Payload	{"title": "Maintenance Notice", "description": "Water supply will be interrupted tomorrow.", "priority": "high"}
Expected Result	200 OK, {"success": true, "data": {"title": "Maintenance Notice", ...}}
Actual Result	200 OK, announcement created; all residents receive notification
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_committee_can_create_announcement(self, client: AsyncClient, seeded_db: Session):
    _, committee_token = create_test_user(seeded_db, RoleEnum.COMMITTEE.name, email="committee_ann@test.com")
    response = await client.post(
        "/api/v1/announcements",
        headers={"Authorization": f"Bearer {committee_token}"},
        json={"title": "Maintenance Notice", "description": "Water supply will be interrupted tomorrow.", "priority": "high"},
    )
    assert response.status_code == status.HTTP_200_OK
    data = response.json()
    assert data["data"]["title"] == "Maintenance Notice"
```

Test Case 9 - Resident Votes in Poll

User Story: US-18 (Resident participates in polls)	
Field	**Value**
API	POST /api/v1/polls/{poll_id}/vote
Test ID / Evidence	test_community.py::TestCommunity::test_resident_can_vote
Input Payload	{"option_id": 1} (Poll "Best facility?" with options "Gym" and "Pool" already created by Committee)
Expected Result	200 OK, {"success": true, "data": {"option_id": 1, ...}}
Actual Result	200 OK, vote recorded; results update live
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_resident_can_vote(self, client: AsyncClient, seeded_db: Session):
    _, committee_token = create_test_user(seeded_db, RoleEnum.COMMITTEE.name, email="committee_vote@test.com")
    poll_resp = await client.post(
        "/api/v1/polls",
        headers={"Authorization": f"Bearer {committee_token}"},
        json={
            "question": "Best facility?",
            "options": [{"option_text": "Gym"}, {"option_text": "Pool"}],
            "start_date": "2026-08-01",
            "end_date": "2026-12-31",
            "is_active": True,
        },
    )
    poll_id = poll_resp.json()["data"]["poll_id"]
    option_id = poll_resp.json()["data"]["options"][0]["option_id"]
    _, profile, resident_token = create_test_resident(seeded_db, flat_number="Vote-101", email="vote@test.com")
    response = await client.post(
        f"/api/v1/polls/{poll_id}/vote",
        headers={"Authorization": f"Bearer {resident_token}"},
        json={"option_id": option_id},
    )
    assert response.status_code == status.HTTP_200_OK
    assert response.json()["data"]["option_id"] == option_id
```

Test Case 10 - AI Staff Recommendations

User Story: US-20 (AI staff recommendations)	
Field	**Value**
API	GET /api/v1/ai/complaints/{complaint_id}/staff-recommendations
Test ID / Evidence	test_ai.py::TestAI::test_staff_recommendations
Input Payload	None (query param: complaint_id in path)
Expected Result	200 OK, {"success": true, "data": {"recommendations": [{"staff_id": 1, "reason": "Electrical specialist"}]}}
Actual Result	200 OK, AI-ranked staff recommendations returned based on specialization, experience, and availability
Execution Status	**PASS**

```python
@pytest.mark.asyncio
async def test_staff_recommendations(self, client: AsyncClient, seeded_db, monkeypatch):
    monkeypatch.setattr(settings, "AI_ENABLED", True)
    monkeypatch.setattr(settings, "GROQ_API_KEY", "test-key")
    monkeypatch.setattr(settings, "GROQ_MODEL", "llama-test")
    cat = ComplaintCategory(category_name="Electrical")
    seeded_db.add(cat)
    seeded_db.commit()
    seeded_db.refresh(cat)
    _resident, profile, _rtoken = create_test_resident(seeded_db)
    complaint = Complaint(
        resident_id=profile.resident_id,
        category_id=cat.category_id,
        description="Sparks from the main switchboard",
        priority="critical",
        status="open",
    )
    seeded_db.add(complaint)
    seeded_db.commit()
    seeded_db.refresh(complaint)
    maint_user, _ = create_test_user(seeded_db, RoleEnum.MAINTENANCE.name, email="elec_ai@example.com")
    staff = MaintenanceStaff(
        user_id=maint_user.user_id,
        specialization="Electrical",
        experience_years=8,
        is_available=True,
    )
    seeded_db.add(staff)
    seeded_db.commit()
    seeded_db.refresh(staff)
    admin, token = create_test_user(seeded_db, RoleEnum.ADMIN.name, email="admin2_ai@example.com")
    with patch(
        "app.services.ai_service.chat_completion",
        return_value=(
            '{"rankings": [{"staff_id": %d, "reason": "Electrical specialist"}]}' % staff.staff_id,
            20,
        ),
    ):
        resp = await client.get(
            f"/api/v1/ai/complaints/{complaint.complaint_id}/staff-recommendations",
            headers={"Authorization": f"Bearer {token}"},
        )
    assert resp.status_code == status.HTTP_200_OK, resp.text
    recs = resp.json()["data"]["recommendations"]
    assert recs[0]["staff_id"] == staff.staff_id
```

User Feedback and Implemented Changes

Feedback from Milestone 3

During Milestone 3 user testing, residents and committee members provided the following

feedback:

Role	Feedback Area	Specific Feedback
Resident	Payment tracking	I want to see my bills and pay them online
Resident	Notifications	I need a single place to see all updates about my complaints, bookings, and visitors
Admin	Announcements	I should be able to publish notices that all residents see immediately
Admin	Maintenance workflow	Staff need a proper way to accept, schedule, and mark tasks complete
Resident	Community engagement	It would be great to vote on community decisions like new facilities
Admin/Committee	AI assistance	Smart suggestions for complaint categories and staff assignment would save time

Changes Implemented in Milestone 4

Feedback	Milestone 4 Implementation	Module	Status
Payment tracking	Added Billing & Payment module with bills, payments, receipts, and summary	Module 8	☑ Complete
Unified notifications	Added Communication & Notification module with announcements and notification inbox	Module 7	☑ Complete
Announcements	Committee/Admin can create, update, delete announcements; auto-notify all residents	Module 7	☑ Complete
Maintenance workflow	Added Maintenance Staff Management with accept/decline/schedule/start/complete task lifecycle	Module 4	☑ Complete
Community engagement	Added Community Engagement module with polls, voting, and live results	Module 9	☑ Complete
AI assistance	Added AI Community Intelligence with Groq-powered classification, staff recommendations, smart search, reports, and chat	Module 10	☑ Complete
Complaint status tracking	Enforced state machine (open → assigned → in_progress → resolved → closed)	Module 3	☑ Complete
Visitor OTP security	Timing-safe HMAC-SHA256 OTP hashing; raw OTP returned once	Module 5	☑ Complete
Facility booking conflicts	Overlap detection, end-before-start validation, maximum duration enforcement	Module 6	☑ Complete
Profile management	Committee can edit resident profiles; residents can upload/delete profile pictures	Module 2	☑ Complete

User Satisfaction (Milestone 4)

Feature	Rating (1-5)	Comments
Complaint Management	5	Real-time tracking with AI classification
Facility Booking	4	Conflict detection works; needs payment integration
Visitor Management	5	OTP system smooth and secure
Notifications	4	Unified inbox; needs SMS/email push
AI Classification	4	Helpful suggestions for category and priority
Admin Dashboard	4	Comprehensive; needs advanced filters
Billing	3	Manual payment entry functional; needs real gateway
Community Polls	4	Easy voting and live results

Conclusion

Milestone 4 represents a complete evolution from Milestone 3, expanding the Neighbourly backend from 30 endpoints across 7 modules to 97 endpoints across 10 modules. All 97 endpoints are fully implemented, documented, and tested with 103 passing pytest cases. The system successfully incorporates user feedback through new modules for billing, notifications, community engagement, maintenance workflows, and AI-powered intelligence. The backend is ready for production deployment pending integration of external services (payment gateway, SMS/email) in future sprints.

--- End of Milestone 4 Report ---